Studio Nx

백엔드 226 — Plugin · PluginPreset 모델

2026-10-18

Backend(FastAPI/Python) 226 — `Plugin` · `PluginPreset`

항목	내용
파일	`studionx/backend/app/models/plugin.py`
작업	S1-09 기본 악기·이펙트: `plugins` · `plugin_presets` 표 (PRD 02 FR-241, FR-244, FR-246)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	225 `setClipNotes` · 227 `TrackInstrument` · `TrackPlugin`

1. 하는 일

가상악기와 이펙트의 **정의**를 담는 표 두 개입니다. 플러그인은 앱에 코드로 박혀 있고, 표에는 그 플러그인을 화면에 어떻게 그릴지 알려 주는 정보만 들어 있습니다.

넣은 것	뜻
`code`	화면(JS)이 구현을 찾는 열쇠. `eq3` 이면 화면의 EQ 구현과 이어집니다 (FR-241·2187)
`name`	사람에게 보이는 이름 (`Subtractive Synth`, `EQ 3-Band` …)
`plugin_type`	`INSTRUMENT`(가상악기) · `EFFECT`(이펙트) · `MIDI_EFFECT`(MIDI 이펙트)
`engine`	소리를 만드는 방식. `WEBAUDIO`(Web Audio 노드 조합) · `WASM`(AudioWorklet, FR-247)
`param_schema`	파라미터 목록(JSON). 이름, 범위, 기본값, 단위, `group`(모달의 칸), `primary`(카드에 노출할 핵심값)가 들어 있고, 카드와 상세 모달이 이 값을 읽어 노브를 자동으로 만듭니다 (FR-244)
`card_asset`	Nyxi 카드 그림 파일 이름 (VARCHAR(100)). 그림은 외부 AI 플랫폼에서 받으므로 지금은 비어 있습니다 (FR-2181)
`is_active`	지금 쓸 수 있는 플러그인인지 (꺼진 것은 목록에서 빠집니다)

`plugin_presets` 는 그 플러그인 위에 붙는 **저장된 파라미터 묶음**입니다. `owner_id` 가 비어 있으면 모두에게 보이는 시스템 프리셋, 값이 있으면 그 사용자가 저장한 개인 프리셋입니다 (FR-246).

넣은 것	뜻
`plugin_id`	어느 플러그인의 프리셋인지 (FK → `plugins`)
`owner_id`	주인 (FK → `users`). 비어 있으면 시스템 기본 프리셋
`name`	프리셋 이름 (`Init`, `Mine` …)
`params`	그 프리셋의 파라미터 값 묶음 (JSON)
`source`	`SYSTEM` · `USER` · `AI`. AI 가 추천해 저장한 것은 `AI` 입니다 (DB 기본값은 `USER`)

2. 언제 쓰나

쓰는 곳	
`plugin_service.list_plugins` · `require_plugin` (232)	플러그인 목록과 한 건을 읽습니다
`plugin_service.list_plugin_presets` (232)	시스템 프리셋과 내 프리셋만 골라 읽습니다

쓰는 곳	
`track_service.add_instrument` · `add_effect` (234·235)	붙이려는 플러그인의 종류를 확인합니다
시드 SQL (242)	기본 악기 2종과 이펙트 5종을 이 표에 넣습니다

3. 코드

종류·방식·출처 (열거형)

```
class PluginType(StrEnum):
    INSTRUMENT = "INSTRUMENT"
    EFFECT = "EFFECT"
    MIDI_EFFECT = "MIDI_EFFECT"


class PluginEngine(StrEnum):
    WEBAUDIO = "WEBAUDIO"
    WASM = "WASM"


class PresetSource(StrEnum):
    SYSTEM = "SYSTEM"
    USER = "USER"
    AI = "AI"
```

`Plugin`

```
class Plugin(Base):
    __tablename__ = "plugins"

    plugin_id: Mapped[int] = mapped_column(INTEGER(unsigned=True), primary_key=True, autoincrement=True)
    code: Mapped[str] = mapped_column(String(50))
    name: Mapped[str] = mapped_column(String(100))
    plugin_type: Mapped[PluginType] = mapped_column(Enum(PluginType))
    engine: Mapped[PluginEngine] = mapped_column(Enum(PluginEngine), server_default=PluginEngine.WEBAUDI
O)
    version: Mapped[str] = mapped_column(String(20), server_default=text("'1.0.0'"))
    param_schema: Mapped[list[dict[str, Any]]] = mapped_column(JSON)
    card_asset: Mapped[str | None] = mapped_column(String(100))
    is_active: Mapped[bool] = mapped_column(Boolean, server_default=text("1"))
```

`PluginPreset` (핵심 칸)

```
    plugin_id: Mapped[int] = mapped_column(ForeignKey("plugins.plugin_id"))
    owner_id: Mapped[int | None] = mapped_column(ForeignKey("users.user_id"))
    name: Mapped[str] = mapped_column(String(100))
    params: Mapped[dict[str, Any]] = mapped_column(JSON)
    source: Mapped[PresetSource] = mapped_column(Enum(PresetSource), server_default=PresetSource.USER)
```

문법 풀이

식	뜻
`StrEnum`	문자열이면서 열거형입니다. `PluginType.INSTRUMENT == "INSTRUMENT"` 가 참이라 DB·JSON 과 그대로 주고받습니다
`Enum(PluginType)`	DB 에는 `ENUM('INSTRUMENT','EFFECT','MIDI_EFFECT')` 로 만들어집니다 (DB.md 와 같습니다)
`server_default=PresetSource.USER`	값을 안 주면 DB 가 `'USER'` 로 채웁니다. DB.md 의 기본값과 같게 맞췄습니다
`Mapped[list[dict[str, Any]]]`	JSON 배열임을 타입으로 알려 줍니다. 실제 값은 `param_schema` 항목의 목록입니다
`owner_id: Mapped[int | None]`	비어 있을 수 있음(NULL)을 뜻합니다. 시스템 프리셋이 이 경우입니다
`passive_deletes=True`	프리셋 삭제를 DB 의 `ON DELETE CASCADE` 에 맡기고 앱이 하나씩 지우지 않습니다

4. 왜 이렇게

선택	이유
`code` 로 화면 구현을 찾음	플러그인마다 화면 코드를 따로 만들면 늘어날수록 관리가 어렵습니다. `code` 하나로 표와 화면을 잇습니다 (FR-241·2187)
그리는 정보를 `param_schema` 에 모음	카드와 상세 모달이 같은 값을 읽어 자동으로 만들어집니다. 플러그인을 추가해도 화면 코드를 새로 쓰지 않습니다 (FR-244·2187)
`owner_id` 로 시스템·개인을 구분	표를 나누지 않고 한 표에서 `owner_id IS NULL`(시스템) 과 `owner_id = 나`(개인) 만 골라 읽습니다
`card_asset` 을 비워 둠	캐릭터 그림은 외부 AI 플랫폼에서 만듭니다. 프로젝트 원칙에 따라 AI 가 그림을 만들지 않습니다 (FR-2181)
`engine` 을 남겨 둠	지금은 Web Audio 로 만들지만, 나중에 WASM(AudioWorklet) 플러그인을 같은 표에 담기 위해서입니다 (FR-247)

5. 시험

`tests/test_plugin_model.py` 에서 두 표의 칸을 실제 MySQL 과 비교(`assert_matches_db`)하고, DB 기본값(`engine` = `WEBAUDIO`, `is_active` = 1, `source` = `USER`), `param_schema` JSON 왕복, 프리셋 `owner_id` 구분을 확인합니다.

> `plugins.code` 는 DB 에서 UNIQUE 입니다. 시드를 두 번 실행해도 `ON DUPLICATE KEY UPDATE` 로 갱신만 됩니다 (242).